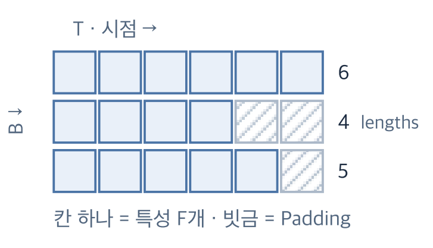
새싹 학습 노트 · Day12 · 2026.11.02 · 순서 모델 · 수강생용 개념 지도

순서 모델 한눈에 보기

RNN → LSTM → Transformer는 "순서를 더 잘 읽기"로 이어지는 한 줄이고, **Autoencoder**는 목적이 다른 갈래입니다. 강의 PDF를 옆에 펴 두고, 막히는 쪽이 지도 어디에 있는지 찾아 읽어 보세요.

세로 모델 하나 **가로** 같은 칸끼리 비교 (강의 p.72 비교표) **주황 화살표** 다음 모델로 넘어간 이유 **Q번호** 아래 질문 목록의 질문·그림·코드

업무 목표와 입력 자료



순서 Tensor `[B, T, F]` 1교시 · 강의 p.4, 18–22

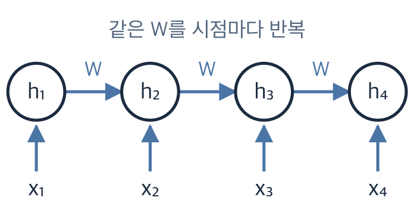
공통 원리: 모든 시점에 같은 가중치를 쓴다. 그래서 길이가 달라도 같은 모델이 그대로 읽는다. 채운 칸(Padding)은 실제 값이 아니므로 모든 모델에서 따로 빼야 한다.

Q1 왜 같은 가중치를 쓰나 · Q2 Padding은 왜 빼나

① 다음 값 예측·분류

순서를 더 잘 읽으려는 한 줄의 발전

RNN 2교시



$h_t = f(h_{t-1}, x_t)$

같은 규칙을 시점마다 반복해 과거를 h 하나에 요약한다

Q3 h는 무엇을 담나 · Q4 output vs h_n

기억: h 하나 (크기 고정)

계산: 순차 · 시간 O(T)

Shape

`output [B,T,H]`

`h_n [L·D,B,H]`

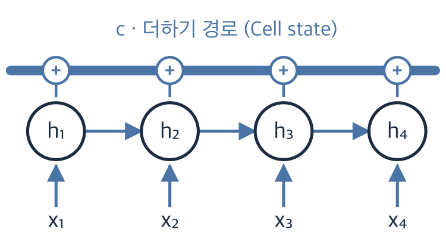
정답 · 지표: 다음 값·클래스 · MSE·Accuracy

강의: **p.23–33 · 코드 p.30**

문제: 같은 W를 계속 곱해 먼 과거가 사라짐 **$0.9^{50} \approx 0.005$**

Q5 왜 사라지나

LSTM 3교시



$c_t = f \odot c_{t-1} + i \odot g$

곱하지 말고 더해서 기억을 잇고, 남길 양은 Gate가 정한다

Q6 왜 더하기 · p.6 계산 · Q7 h와 c는 왜 둘

기억: h + c (장기 기억 c 추가)

계산: 순차 · 시간 O(T)

Shape

`output [B,T,H]`

`(h_n, c_n)` 둘 다 `[L·D,B,H]`

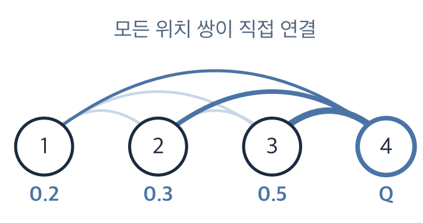
정답 · 지표: 다음 값 · MSE (실습: 과거 12개 → 다음 1개)

강의: **p.34–45 · 코드 p.41**

문제: 여전히 한 줄로 순서대로 **병렬 불가**

Q8 왜 Attention

Transformer Encoder 5–6교시



$$출력 = \Sigma\ a \cdot v$$
$$a = \mathrm{softmax}(q \cdot k / \sqrt{d})$$

필요한 위치를 직접 골라 비율대로 섞는다

Q9 Q·K·V · p.9 계산 · Q10 왜 √d · Q11 왜 Position · Q2 Mask · Q12 한 층 · LayerNorm

기억: 기억 대신 모든 위치를 직접 참조

계산: 병렬 · 점수 T² 개 Q13 T²

Shape

`[B,T,E] → [B,T,E]` 크기 유지

→ 평균 Pooling `[B,E]`

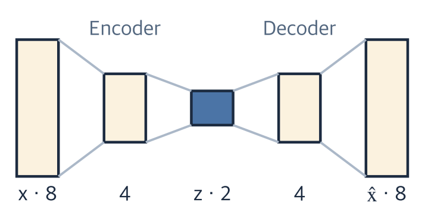
정답 · 지표: 클래스·값 · Accuracy·AUC·MSE

강의: **p.59–75 · 코드 p.67, 75**

② 입력 재구성

목적이 다른 갈래

Autoencoder 4교시



$x \rightarrow z(작게) \rightarrow \hat{x} \approx x$

일부러 좁힌 통로로 복사하게 해서 주요 패턴을 배운다

Q14 왜 따로 떨어져 있나 · Q15 왜 가운데를 좁게 · Q16 오차 큼 = 이상 후보

기억: 시간축 없음 · Latent z에 압축

계산: 샘플마다 독립 · 입력 차원에 의존

Shape

`[B,8]→[B,2]→[B,8]`

샘플별 오차 `[B]`

정답 · 지표: 입력 자신 · 재구성 오차

강의: **p.46–58 · 코드 p.56**

7교시 · 길이 T를 늘리면 (강의 p.88 길이)

T	16	32	64	128
RNN·LSTM 순차 단계	16	32	64	128
Attention 점수 T²	256	1,024	4,096	16,384

길이 2배 → 순차 단계 2배, Attention 점수 4배. 다만 RNN은 단계를 나눠 돌릴 수 없고, Attention은 한꺼번에 계산한다. 실제 측정 시간은 장치·Batch·오버헤드에 따라 달라진다(강의 p.90).

8교시 · 두 갈래 합치기

LSTM 예측 결과와 Autoencoder 재구성 결과는 **목적이 달라 표를 따로** 만든다. 두 지표를 평균 내지 않는다(강의 p.11, 52, 76).

질문 목록

질문과 한 줄 답만 훑어도 오늘의 "왜"가 정리됩니다. 질문마다 그림과 "코드에서는"을 함께 실었습니다. 지도의 Q번호를 누르면 해당 질문으로 이동합니다.

공통 바탕 1교시

Q1 RNN은 왜 모든 시점에 같은 가중치를 쓰나?

시점마다 다른 가중치를 쓰면 길이가 바뀔 때마다 새로 배워야 한다. 같은 가중치를 쓰면 길이가 4든 50이든 같은 모델로 읽는다.

따로	W₁ W₂ W₃ W₄ W₅ ?	길이 5는 못 읽음
같이	W W W W W	길이 상관없음

코드에서는 `nn.RNN(input_size, hidden_size)` 에는 길이 T를 정하는 인자가 없다. 그래서 길이가 다른 입력도 같은 모델에 넣는다 (p.28).

Q2 Padding은 왜 계속 빼라고 하나?

Padding은 길이를 맞추려고 채운 0이라 실제 값이 아니다. 계산에 섞이면 평균이나 마지막 값이 틀어진다.

순서	3 1 4 0 0	실제 길이 3 + Padding 2
×	`(3+1+4+0+0) ÷ 5 = 1.6`	Padding까지 평균
○	`(3+1+4) ÷ 3 = 2.67`	실제 길이로 평균

코드에서는 RNN·LSTM은 `lengths` 로 실제 마지막 시점을 고른다. Transformer는 `src_key_padding_mask` 로 Padding을 가리고, 평균은 실제 길이로 나눈다 (p.67, 77).

RNN 2교시

Q3 Hidden state h에는 무엇이 들어 있나?

지금까지 읽은 입력을 정해진 크기(hidden_size)로 요약한 값이다. 크기가 정해져 있어 다 담지 못하고, 예측에 필요한 것 위주로 남는다.

비유 글을 읽으며 적는 메모 한 장. 칸이 정해져 있어 중요한 것만 적는다.

입력	x_1 x_2 x_3 … x_{50}	길이 4든 50이든
h	h · 16칸	크기는 항상 hidden_size

코드에서는 `nn.LSTM(1, 16)` 의 16이 이 요약의 크기(hidden_size)다 (p.41).

Q4 output과 h_n은 뭐가 다른가?

output은 모든 시점의 Hidden state를 모은 것이고, h_n은 마지막 시점의 Hidden state다.

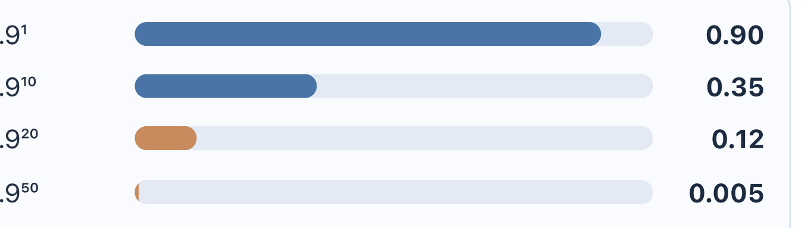
output	h_1 h_2 h_3 h_4	`[B,T,H]`
h_n	h_1 h_2 h_3 h_4	`[L·D,B,H]`

코드에서는 분류 모델은 `h_n[-1]` (p.30), 예측 모델은 `output[:, -1]` (p.41)로 마지막 시점을 꺼낸다. 1층·단방향·Padding 없음이면 둘은 같은 값이다.

LSTM 3교시

Q5 RNN은 왜 긴 순서를 잘 기억하지 못하나?

같은 가중치를 시점마다 계속 곱하기 때문이다. 1보다 작은 수를 여러 번 곱하면 0에 가까워지듯, 먼 과거에서 오는 Gradient가 사라진다.



0.9^{1}	0.90
0.9^{10}	0.35
0.9^{20}	0.12
0.9^{50}	0.005

곱하는 횟수(시점 수)가 늘수록 거의 0이 된다.

코드에서는 그래서 긴 Window에는 `nn.RNN` 대신 `nn.LSTM` 을 쓴다. 실습 03 응용에서 둘의 valid MSE를 비교한다 (p.44).

Q6 LSTM은 기억을 어떻게 오래 유지하나?

이전 Cell state에 남길 비율(forget gate)을 곱하고, 새 정보는 쓸 비율(input gate)만큼 더한다. 곱하기만 반복하지 않고 더하기로 이어 가서 기억이 오래 간다.

남길 양: 이전 c 2 × forget 0.5 = 1

쓸 양: 새 후보 0.5 × input 0.4 = 0.2

새 c: 1 + 0.2 = 1.2 $c = f \odot c + i \odot g$ · 강의 p.6

코드에서는 Gate 계산은 `nn.LSTM` 이 모두 알아서 한다. 우리가 정하는 것은 `hidden_size` 와 층 수 정도다 (p.36).

Q7 LSTM은 왜 상태가 h와 c 두 개인가?

Cell state c는 안에서 오래 들고 가는 기억이고, Hidden state h는 그중 지금 필요한 부분을 꺼낸 출력이다.

c: Cell state —— 안에서만 전달

h: Hidden state → output 밖으로 나감

코드에서는 `output, (h_n, c_n) = lstm(x)` 처럼 h와 c를 괄호로 묶어 따로 돌려준다. 둘을 하나의 Tensor로 다루면 오류가 난다 (p.40, 43).

Transformer 5–6교시

Q8 LSTM이 있는데 왜 Attention이 또 나왔나?

LSTM도 한 시점씩 차례로 계산해서 느리고, 먼 시점의 정보는 중간을 모두 거쳐 와야 한다. Attention은 모든 위치를 한 번에 직접 비교한다.

LSTM: 1 → 2 → 3 → 4 차례로 · 거쳐 옴

Attention: 1 2 3 4 4가 1·2·3을 한 번에 직접 봄

코드에서는 그래서 Transformer는 시점을 하나씩 돌지 않고 `[B,T,E]` 전체를 한 번에 받는다 (p.64).

Q9 Attention은 무엇을 계산하나? (Query·Key·Value)

Query와 Key를 비교해 어느 위치를 얼마나 볼지(Attention weight)를 정하고, 그 비율대로 Value를 섞는다.

비유 회의에서 내 질문(Query)과 관련 있는 사람(Key)일수록 그 사람 말(Value)에 더 귀 기울인다.

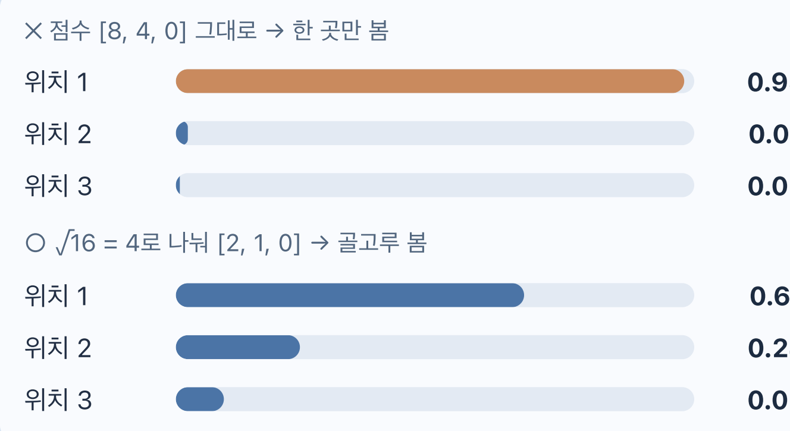
weight	0.2 0.3 0.5	Query·Key 비교 → softmax, 합 1
Value	10 20 30	섞을 내용
출력	`0.2×10 + 0.3×20 + 0.5×30 = 23`	강의 p.9

코드에서는 Self-Attention은 Query·Key·Value를 모두 같은 입력에서 만들어서 `attn(x, x, x)` 로 쓴다 (p.62).

Q10 Attention 점수는 왜 √d로 나눠서 쓰나?

점수 차이가 너무 크면 softmax가 한 위치에 거의 100%를 몰아준다. 그러면 학습이 잘 안 되므로, 점수를 줄여 골고루 보게 한다.

비유 점수 차이가 너무 크면 한 사람 말만 듣게 된다.



× 점수 [8, 4, 0] 그대로 → 한 곳만 봄

위치 1	0.98
위치 2	0.02
위치 3	0.00

○ √16 = 4로 나눠 [2, 1, 0] → 골고루 봄

위치 1	0.67
위치 2	0.24
위치 3	0.09

코드에서는 직접 쓸 일은 없다. `nn.MultiheadAttention` 안에서 자동으로 나눈다.

Q11 Transformer에는 왜 Position을 더하나?

Attention은 위치끼리 비교만 해서 순서를 모른다. 단어 순서를 바꿔도 같은 묶음으로 본다. 그래서 위치마다 다른 값을 더해 순서를 알려 준다.

문장 1	woman ate raccoon	
문장 2	raccoon ate woman	뜻은 반대인데 같은 묶음
해결	입력 + Position	위치 1·2·3마다 다른 값

코드에서는 `x + self.position(pos)` 처럼 입력에 Position Embedding을 더한다. 그래서 입력 길이는 `max_len` 이하여야 한다 (p.75).

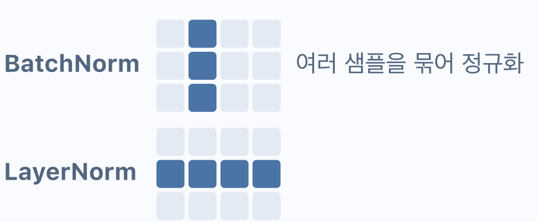
Q12 Encoder Layer 한 층 안에는 무엇이 있나?

Self-Attention(위치끼리 정보 섞기)과 Feedforward(위치마다 따로 변환)가 있고, 각각 뒤에 Residual 더하기와 LayerNorm이 붙는다. 입력과 출력 크기가 같아서 여러 층을 쌓을 수 있다.

한 층

Self-Attention + LayerNorm → Feedforward + LayerNorm

크기 그대로 `[B,T,E]`



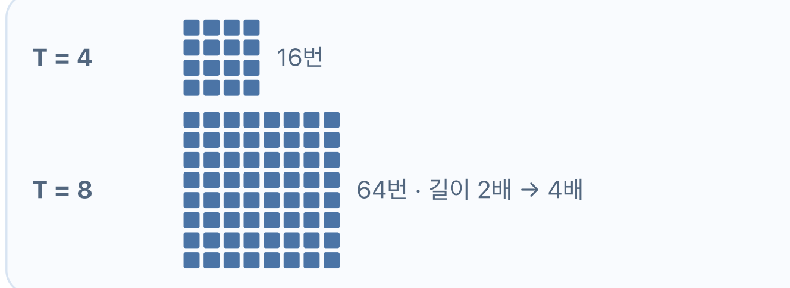
BatchNorm 여러 샘플을 묶어 정규화

위치 하나 안에서 정규화 → 길이가 제각각이어도 OK

코드에서는 `nn.TransformerEncoderLayer(d_model=16, nhead=4)` 하나가 이 한 층이다. `d_model` 은 `nhead` 로 나누어떨어져야 한다 (p.64, 73).

Q13 순서가 길어지면 왜 Transformer가 더 무거워지나?

모든 위치 쌍을 비교하므로 길이가 T면 T×T번 비교한다. 길이가 2배면 비교는 4배가 된다.



코드에서는 실습 07에서 길이 16·32·64로 시간과 Peak Memory를 재 보면 이 차이가 보인다 (p.88, 91).

Autoencoder 4교시

Q14 Autoencoder는 왜 따로 떨어져 있나?

미래 값을 맞히는 게 아니라 입력을 그대로 되살리는 게 목표라서, 정답과 평가 지표가 다르다.

예측	과거 12개 → 다음 1개	정답 = 미래 값
재구성	x 8개 → x̂ 8개	정답 = x 자신

코드에서는 정답이 입력 자신이라 `loss = loss_fn(recon, x)` 처럼 x와 비교한다 (p.56).

Q15 Autoencoder는 왜 가운데를 좁게 만드나?

가운데(Latent)가 넓으면 입력을 그대로 베끼기만 한다. 좁게 만들어야 중요한 특징만 골라 담는다. 너무 좁으면 중요한 정보까지 잃는다.

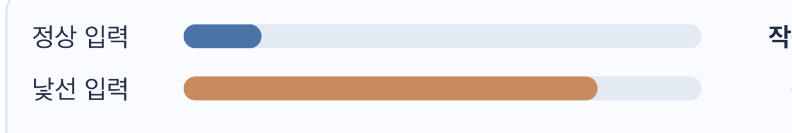
비유 긴 글을 세 줄로 요약했다가 다시 풀어 쓰기. 요약이 짧아야 핵심만 남는다.

통로	8 → 4 → 2 → 4 → 8
너무 넓으면	그대로 베낌 · 배운 것 없음
너무 좁으면	중요한 정보도 잃음

코드에서는 `nn.Linear(4, 2)` 의 2가 Latent 크기다. 실습 04 응용에서 1·2·4를 비교한다 (p.56–57).

Q16 재구성 오차가 크면 왜 '이상'이 아니라 '이상 후보'인가?

정상 데이터로만 배운 모델이라 낯선 입력은 잘 되살리지 못해 오차가 커진다. 하지만 전처리나 모델 크기 때문에도 오차가 커질 수 있어 확정할 수는 없다.



정상 입력	작음
낯선 입력	큼

개념 그림이다. 기준값(임계값)은 valid에서 정한다.

코드에서는 `MSELoss(reduction="none")` 로 샘플별 오차를 구하고, 기준값은 valid에서 정한 뒤 test는 마지막에 한 번만 본다 (p.54–55).

NVIDIA AI 코어 엔지니어 2기 · Day12 딥러닝 핵심 아키텍처 및 학습 · 강의 PDF `20261102-2.pdf`(103쪽) 범위. 본문의 p.번호는 강의 PDF 쪽입니다.

참고 교재: GF = Goodfellow, Bengio, Courville, *Deep Learning*, MIT Press, 2016. UDL = Simon J. D. Prince, *Understanding Deep Learning*, MIT Press, 2023.

질문별 근거: Q1 GF 10장 도입 · UDL 12.1 · Q2 UDL 12.7 (Mask) · Q3 GF 10.1 · Q4 GF 10.2 · Q5 GF 10.7 · Q6 GF 10.9.2, 10.10 · Q7 GF 10.10.1 · Q8 GF 10.2 · UDL 12장 Notes · Q9 UDL 12.2 · Q10 UDL 12.3.2 · Q11 UDL 12.3.1 · Q12 UDL 12.3.3, 12.4, 11.4 Notes · Q13 UDL 12.2.1, 12.9 · Q14 강의 p.52, 72 · Q15 GF 14장 도입, 14.1–14.2 · Q16 GF 14장 도입.

비유는 이해를 돕기 위한 것으로 실제 계산과 같지 않습니다.